Advanced Machine Learning and Data Analysis for the Physical Sciences

Morten Hjorth-Jensen[1]

Department of Physics and Center for Computing in Science Education, University of Oslo, Norway[1]

April 30, 2026

© 1999-2026, Morten Hjorth-Jensen. Released under CC Attribution-NonCommercial 4.0 license

Plans for the week of April 27 – May 1, 2026

Deep Generative Models: from VAEs to Diffusion

1. Reminder: Variational Autoencoders (VAEs) — key equations and limitations
2. From VAEs to hierarchical VAEs and the diffusion limit
3. Mathematical derivation of diffusion models (DDPM)
4. The noise-prediction objective and connection to score matching
5. Reverse diffusion and the denoising network
6. DDPM PyTorch implementation on MNIST
7. VAEs vs. Diffusion Models: pros, cons, and when to use each

Readings

Core references

1. Kingma & Welling, *Auto-Encoding Variational Bayes* (2014), arXiv:1312.6114
2. Sohl-Dickstein et al., *Deep Unsupervised Learning using Nonequilibrium Thermodynamics* (2015), arXiv:1503.03585
3. Ho, Jain & Abbeel, *Denoising Diffusion Probabilistic Models* (2020), arXiv:2006.11239
4. Calvin Luo, *Understanding Diffusion Models: A Unified Perspective*, arXiv:2208.11970 (the main source for the derivations here)
5. Kingma et al., *Variational Diffusion Models*, arXiv:2107.00630
6. Foster, *Generative Deep Learning*, ch. 4 (VAEs) and ch. 8 (Diffusion)

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

Generative models: the common thread

All generative models we study share the same goal: learn a distribution $p_\theta(\mathrm{x})$ over data x such that we can both *evaluate* the density and *sample* from it.

Three major families (this course)

Model	**Latent space**	**Key mechanism**
Energy-based / RBM	fixed-dim h	Contrastive Divergence
VAE	fixed-dim h	ELBO maximization
Diffusion	**same dim as** x	**Iterative denoising**

Today we derive diffusion models *starting* from the VAE framework, showing they can be understood as VAEs taken to a Markovian limit with infinitely many latent hierarchies.

VAE: Architecture Recap

An autoencoder learns a mapping $\mathrm{x} \to \mathrm{h} \to \hat{\mathrm{x}}$ with $\|\mathrm{x} - \hat{\mathrm{x}}\|^2$ as loss. A **variational** autoencoder replaces the deterministic encoder with a *distribution*:

Encoder (inference network):

$$q_\phi(\mathrm{h}|\mathrm{x}) = \mathcal{N}\big(\mathrm{h};\, \boldsymbol{\mu}_\phi(\mathrm{x}),\, \boldsymbol{\sigma}^2_\phi(\mathrm{x})\mathrm{I}\big)$$

Decoder (generative network):

$$p_\theta(\mathrm{x}|\mathrm{h})$$

Prior:

$$p(\mathrm{h}) = \mathcal{N}(\mathrm{h}; 0, \mathrm{I})$$

Goal

Maximize the marginal log-likelihood

$$\log p_\theta(\mathrm{x}) = \log \int p_\theta(\mathrm{x}, \mathrm{h})\, d\mathrm{h}$$

which is intractable $\to$ variational lower bound.

The ELBO derivation (from week 13)

For any approximate posterior $q_\phi(\mathrm{h}|\mathrm{x})$:

$$\begin{aligned}
\log p(\mathrm{x}) &= \log \int \frac{p(\mathrm{x},\mathrm{h})}{q_\phi(\mathrm{h}|\mathrm{x})}\, q_\phi(\mathrm{h}|\mathrm{x})\, d\mathrm{h} \\
&= \log \mathbb{E}_{q_\phi(\mathrm{h}|\mathrm{x})}\left[\frac{p(\mathrm{x},\mathrm{h})}{q_\phi(\mathrm{h}|\mathrm{x})}\right] \\
&\geq \underbrace{\mathbb{E}_{q_\phi(\mathrm{h}|\mathrm{x})}\left[\log \frac{p(\mathrm{x},\mathrm{h})}{q_\phi(\mathrm{h}|\mathrm{x})}\right]}_{\mathcal{L}(\phi,\boldsymbol{\theta})} \quad \text{(Jensen's inequality)}
\end{aligned}$$

Splitting the joint:

$$\boxed{\mathcal{L}(\phi,\boldsymbol{\theta}) = \underbrace{\mathbb{E}_{q_\phi(\mathrm{h}|\mathrm{x})}[\log p_{\boldsymbol{\theta}}(\mathrm{x}|\mathrm{h})]}_{\text{Reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q_\phi(\mathrm{h}|\mathrm{x}) \,\|\, p(\mathrm{h})\big)}_{\text{Regularisation}}}$$

VAE: ELBO interpretation

$$\log p(\mathrm{x}) = \underbrace{\mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta})}_{\leq \log p(\mathrm{x})} + \underbrace{D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\mathrm{h}|\mathrm{x}) \,\|\, p(\mathrm{h}|\mathrm{x})\big)}_{\geq 0}$$

- Since $D_{\mathrm{KL}} \geq 0$, the ELBO is a **lower bound** on the log-evidence.
- Maximising the ELBO simultaneously:
  1. maximises the reconstruction quality, and
  2. minimises the gap to the true posterior.
- The ELBO is tight ($= \log p(\mathrm{x})$) only when $q_{\boldsymbol{\phi}}(\mathrm{h}|\mathrm{x}) = p(\mathrm{h}|\mathrm{x})$.

VAE: the reparameterisation trick

To back-propagate through a *sampling* node we write

$$\mathsf{h} = \boldsymbol{\mu}_\phi(\mathsf{x}) + \boldsymbol{\sigma}_\phi(\mathsf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathsf{I}).$$

The gradient $\nabla_\phi \mathbb{E}_{q_\phi(\mathsf{h}|\mathsf{x})}[f(\mathsf{h})]$ can now be estimated as $\mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(0,\mathsf{I})}[\nabla_\phi f(\boldsymbol{\mu}_\phi(\mathsf{x}) + \boldsymbol{\sigma}_\phi(\mathsf{x}) \odot \boldsymbol{\epsilon})]$, which is low-variance and unbiased.

Closed-form KL for Gaussian encoder and prior

When $q = \mathcal{N}(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2 \mathsf{I})$ and $p(\mathsf{h}) = \mathcal{N}(0, \mathsf{I})$:

$$D_{\mathrm{KL}}(q \,\|\, p) = \frac{1}{2} \sum_{j=1}^{d_h} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\right)$$

This term regularises the latent code, preventing posterior collapse to a Dirac delta.

VAE: Strengths and Limitations

Strengths

- Principled probabilistic framework
- Continuous, structured latent space
- Efficient single-pass inference
- Stable training (no adversary)
- Exact ELBO evaluation
- Interpretable disentangled representations

Limitations

- **Blurry samples** — Gaussian decoder averages over modes
- **Posterior collapse** — powerful decoder ignores h
- Fixed-dim latent bottle-neck
- Gaussian prior may not fit complex data
- ELBO gap: $\log p(\mathrm{x})$ is never exactly recovered
- Limited expressiveness of mean-field approximation

These limitations motivate going beyond a single VAE — either by stacking them (hierarchical VAEs) or taking the diffusion limit.

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

Hierarchical (Markovian) VAEs

We can generalise the single-latent VAE to a **chain of T latent spaces** $\mathsf{h}_1, \mathsf{h}_2, \ldots, \mathsf{h}_T$ with Markovian structure:

$$p(\mathsf{x}, \mathsf{h}_{1:T}) = p(\mathsf{h}_T)\, p_\theta(\mathsf{x}|\mathsf{h}_1) \prod_{t=2}^{T} p_\theta(\mathsf{h}_{t-1}|\mathsf{h}_t) \quad \text{(generative)}$$

$$q_\phi(\mathsf{h}_{1:T}|\mathsf{x}) = q_\phi(\mathsf{h}_1|\mathsf{x}) \prod_{t=2}^{T} q_\phi(\mathsf{h}_t|\mathsf{h}_{t-1}) \quad \text{(inference)}$$

The ELBO for a Markovian hierarchical VAE becomes:

$$\log p(\mathsf{x}) \geq \mathbb{E}_{q_\phi(\mathsf{h}_{1:T}|\mathsf{x})} \left[\log \frac{p(\mathsf{h}_T)\, p_\theta(\mathsf{x}|\mathsf{h}_1) \prod_{t=2}^{T} p_\theta(\mathsf{h}_{t-1}|\mathsf{h}_t)}{q_\phi(\mathsf{h}_1|\mathsf{x}) \prod_{t=2}^{T} q_\phi(\mathsf{h}_t|\mathsf{h}_{t-1})} \right]$$

Each level adds a new reconstruction/regularisation trade-off.

Three restrictions that give a VDM

A **Variational Diffusion Model** (VDM) is a Markovian hierarchical VAE subject to three additional constraints (Luo 2022):

Constraint 1: Full-dimensional latents

$\dim(\mathrm{h}_t) = \dim(\mathrm{x}_0)$ for all t. The latent variable $\mathrm{h}_t \equiv \mathrm{x}_t$ lives in *data space*, not a compressed bottleneck.

Constraint 2: Fixed (non-learned) Gaussian encoder

$$q(\mathrm{x}_t|\mathrm{x}_{t-1}) = \mathcal{N}\big(\mathrm{x}_t;\ \sqrt{\alpha_t}\,\mathrm{x}_{t-1},\ (1-\alpha_t)\mathrm{I}\big)$$

The encoder is not trained; it is a fixed noise schedule.

Constraint 3: Terminal distribution is standard Gaussian

α_t is chosen so that as $t \to T$, $q(\mathrm{x}_T|\mathrm{x}_0) \approx \mathcal{N}(0, \mathrm{I})$.

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

The forward (noising) process q

Define cumulative noise levels $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. By the Markov structure, there is a *closed-form* for the marginal at any time t given only x_0:

Closed-form forward diffusion (key result)

$$\boxed{q(\mathsf{x}_t|\mathsf{x}_0) = \mathcal{N}\big(\mathsf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathsf{x}_0,\ (1-\bar{\alpha}_t)\mathsf{I}\big)}$$

Proof (induction): Starting from $q(\mathsf{x}_t|\mathsf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\,\mathsf{x}_{t-1}, (1-\alpha_t)\mathsf{I})$, the Gaussian convolution formula gives:

$$\begin{aligned}
\mathsf{x}_t &= \sqrt{\alpha_t}\,\mathsf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_{t-1} \\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\mathsf{x}_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1}) + 1 - \alpha_t}\,\bar{\boldsymbol{\epsilon}} \\
&= \cdots = \sqrt{\bar{\alpha}_t}\,\mathsf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}
\end{aligned}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathsf{I})$.

The forward process: noise schedule

Two common schedules for $\beta_t = 1 - \alpha_t$:
Linear (Ho et al. 2020):

$$\beta_t = \beta_{\text{min}} + \frac{t-1}{T-1}(\beta_{\text{max}} - \beta_{\text{min}})$$

with $\beta_{\text{min}} = 10^{-4}$, $\beta_{\text{max}} = 0.02$, $T = 1000$.
Cosine (Nichol & Dhariwal 2021):

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)$$

avoids near-zero SNR at the start.

Signal-to-noise ratio (SNR)

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}$$

$\mathrm{SNR}(0) \gg 1$ (clean), $\mathrm{SNR}(T) \approx 0$ (pure noise). The VDM objective can be rewritten entirely in terms of SNR.

Posterior of forward process: $q(\mathrm{x}_{t-1}|\mathrm{x}_t, \mathrm{x}_0)$

The reverse of the forward step conditioned on x_0 is tractable:

Forward posterior (exact)

$$q(\mathrm{x}_{t-1}|\mathrm{x}_t, \mathrm{x}_0) = \mathcal{N}\big(\mathrm{x}_{t-1};\ \tilde{\boldsymbol{\mu}}_t(\mathrm{x}_t, \mathrm{x}_0),\ \tilde{\beta}_t \mathrm{I}\big)$$

with

$$\tilde{\boldsymbol{\mu}}_t(\mathrm{x}_t, \mathrm{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathrm{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathrm{x}_t$$

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t$$

This is the *target* for the learned reverse process $p_{\boldsymbol{\theta}}(\mathrm{x}_{t-1}|\mathrm{x}_t)$.

Optimising the VDM: ELBO decomposition

Substituting the VDM structure into the MHVAE ELBO and simplifying:

$$\log p(\mathrm{x}_0) \geq \underbrace{\mathbb{E}_{q(\mathrm{x}_1|\mathrm{x}_0)}[\log p_\theta(\mathrm{x}_0|\mathrm{x}_1)]}_{\mathcal{L}_0\text{: reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q(\mathrm{x}_T|\mathrm{x}_0) \,\|\, p(\mathrm{x}_T)\big)}_{\mathcal{L}_T\text{: prior matching, no params}} - \underbrace{\sum_{t=2}^{T} \mathbb{E}_{q(\mathrm{x}_t|\mathrm{x}_0)}\big[D_{\mathrm{KL}}\big(q(\mathrm{x}_{t-1}|\mathrm{x}_t, \mathrm{x}_0) \,\|\, p_\theta(\mathrm{x}_{t-1}|\mathrm{x}_t)\big)\big]}_{\mathcal{L}_{1:T-1}\text{: denoising matching terms}}$$

$\mathcal{L}_T = 0$ by design. $\mathcal{L}_0$ can be treated as a single decoder step. The dominant cost is in $\mathcal{L}_{1:T-1}$.

Reducing variance: conditioning on x_0

The **consistency term** in the first derivation involved two random variables per step, giving high variance.
Conditioning on x_0 at each step (using the Markov property):

$$q(x_{t-1}|x_t, x_0) \text{ replaces } q(x_{t-1}|x_t)$$

reduces each KL term to an expectation over *one* random variable $x_t \sim q(x_t|x_0)$, substantially lowering Monte Carlo variance.

The improved ELBO

Each denoising matching term is:

$$\mathcal{L}_{t-1} = \mathbb{E}_{q(x_t|x_0)}\left[D_{\mathrm{KL}}\big(q(x_{t-1}|x_t, x_0) \,\|\, p_\theta(x_{t-1}|x_t)\big)\right]$$

Since both distributions are Gaussian, the KL has a closed form.

Parameterising the reverse process

Choose $p_{\boldsymbol{\theta}}(\mathrm{x}_{t-1}|\mathrm{x}_t) = \mathcal{N}(\mathrm{x}_{t-1}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathrm{x}_t, t), \tilde{\beta}_t \mathrm{I})$.
The KL between two Gaussians with equal variance reduces to a squared mean difference:

$$\mathcal{L}_{t-1} = \mathbb{E}_{q(\mathrm{x}_t|\mathrm{x}_0)}\left[\frac{1}{2\tilde{\beta}_t}\left\|\tilde{\boldsymbol{\mu}}_t(\mathrm{x}_t, \mathrm{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\right\|^2\right] + C$$

We can further reparameterise $\tilde{\boldsymbol{\mu}}_t$ using $\mathrm{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathrm{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}\right)$:

$$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}\left(\mathrm{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}\right)$$

If the network predicts $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) \approx \boldsymbol{\epsilon}$:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathrm{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\right)$$

The noise-prediction objective (DDPM)

Substituting into $\mathcal{L}_{t-1}$:

$$\mathcal{L}_{t-1} = \mathbb{E}_{\mathsf{x}_0,\boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\tilde{\beta}_t\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathsf{x}_t, t)\right\|^2\right]$$

Ho et al. (2020) showed that **dropping the weight** and averaging over t gives a simpler, better-performing objective:

DDPM training loss

$$\boxed{\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,\mathsf{x}_0,\boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\sqrt{\bar{\alpha}_t}\,\mathsf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t)\right\|^2\right]}$$

where $t \sim \mathrm{Uniform}[1, T]$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathsf{I})$.

The network $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$ **predicts the noise** that was added.

Connection to score matching

The noise prediction objective has a deep connection to **score matching**.
The *score* of a distribution is $\nabla_{\mathrm{x}} \log p(\mathrm{x})$.
For the forward-diffused data at time t:

$$\nabla_{\mathrm{x}_t} \log q(\mathrm{x}_t|\mathrm{x}_0) = -\frac{\mathrm{x}_t - \sqrt{\bar{\alpha}_t}\,\mathrm{x}_0}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}}$$

Therefore predicting $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) \approx \boldsymbol{\epsilon}$ is equivalent to estimating the *score*:

$$\mathrm{s}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) = -\frac{\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{\mathrm{x}_t} \log q(\mathrm{x}_t|\mathrm{x}_0)$$

This links DDPM to **score-based generative models** (Song & Ermon, 2019) and **stochastic differential equations** (Song et al. 2021).

The reverse sampling algorithm (DDPM)

Given the trained denoiser $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$:

Algorithm: DDPM reverse sampling

1. Sample $\mathrm{x}_T \sim \mathcal{N}(0, \mathrm{I})$.
2. For $t = T, T-1, \ldots, 1$:
   1. Sample $\mathrm{z} \sim \mathcal{N}(0, \mathrm{I})$ if $t > 1$, else $\mathrm{z} = 0$.
   2. Compute $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathrm{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\right)$
   3. $\mathrm{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathrm{x}_t, t) + \sqrt{\tilde{\beta}_t}\,\mathrm{z}$
3. Return x_0.

Each step is one neural network evaluation + cheap Gaussian update. With $T = 1000$ steps this is slow but produces high-quality samples.

Three equivalent parameterisations

The denoising network can predict three equivalent targets, each with slightly different properties:

Predict	**Objective**	**Remark**
$\boldsymbol{\epsilon}$ (noise)	$\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\|^2$	DDPM; most comm
x_0 (data)	$\|\mathrm{x}_0 - \hat{\mathrm{x}}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\|^2$	Image prior; can ove
$\mathrm{v} = \sqrt{\bar{\alpha}_t}\,\boldsymbol{\epsilon} - \sqrt{1-\bar{\alpha}_t}\,\mathrm{x}_0$ (velocity)	$\|\mathrm{v} - \mathrm{v}_{\boldsymbol{\theta}}(\mathrm{x}_t, t)\|^2$	Progressive Distillat

The velocity parameterisation (Salimans & Ho, 2022) is particularly useful for few-step distillation and stable training at small t.

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

PyTorch DDPM on MNIST — Overview

This self-contained implementation covers:

1. Imports and hyperparameters / noise schedule
2. Data loading (MNIST, normalised to $[-1, 1]$)
3. Lightweight U-Net with sinusoidal time embedding
4. Forward diffusion: `q_sample`
5. Training loss: MSE between predicted and true noise
6. Training loop
7. Reverse sampling: `p_sample_loop`

References

Jackson-Kang's PyTorch diffusion tutorial (GitHub) and awjuliani's DDPM (GitHub).

Imports and noise schedule

```python
import torch, torch.nn as nn, torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt, math

device = 'cuda' if torch.cuda.is_available() else 'cpu'

# --- Hyperparameters ---------------------------------------------------
T           = 300        # diffusion timesteps
beta_start  = 1e-4
beta_end    = 0.02
img_size    = 28
channels    = 1
batch_size  = 128
epochs      = 5
lr          = 2e-4

# --- Linear noise schedule ---------------------------------------------
betas           = torch.linspace(beta_start, beta_end, T, device=device)
alphas          = 1.0 - betas
alphas_cumprod  = torch.cumprod(alphas, dim=0)   # \bar{\alpha}_t
```

Sinusoidal time embedding

A key ingredient is conditioning the denoiser on the timestep t. We use sinusoidal embeddings (analogous to transformer positional encoding):

```
class SinusoidalPosEmb(nn.Module):
    """Map scalar t in [0,1] to a d-dimensional embedding."""
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):           # t: (B,) in [0, 1]
        half = self.dim // 2
        freqs = torch.exp(
            -math.log(10000) *
            torch.arange(half, device=t.device) / (half - 1)
        )
        args  = t.unsqueeze(1) * freqs.unsqueeze(0)   # (B, half)
        emb   = torch.cat([args.sin(), args.cos()], dim=-1)
        return emb                                     # (B, dim)
```

Lightweight U-Net model

```python
class SimpleUNet(nn.Module):
    def __init__(self, c, emb_dim=128):
        super().__init__()
        self.time_mlp = nn.Sequential(
            SinusoidalPosEmb(emb_dim),
            nn.Linear(emb_dim, emb_dim), nn.ReLU()
        )
        self.enc1 = nn.Conv2d(c,   64,  3, padding=1)
        self.enc2 = nn.Conv2d(64, 128,  3, padding=1)
        self.dec1 = nn.ConvTranspose2d(128, 64, 3, padding=1)
        self.dec2 = nn.ConvTranspose2d(64,  c,  3, padding=1)
        self.act  = nn.ReLU()
        self.proj = nn.Linear(emb_dim, 128)  # time -> channel

    def forward(self, x, t):   # x: (B,C,H,W), t: (B,) in [0,1]
        temb = self.proj(self.time_mlp(t)).view(-1, 128, 1, 1)
        h    = self.act(self.enc1(x))
        h    = self.act(self.enc2(h)) + temb
        h    = self.act(self.dec1(h))
        return self.dec2(h)
```

Forward diffusion and training loss

```python
def q_sample(x0, t, noise=None):
    """
    Sample x_t = sqrt(alpha_bar_t) * x0 + sqrt(1-alpha_bar_t) * eps
    Implements the closed-form forward process q(x_t | x_0).
    """
    if noise is None:
        noise = torch.randn_like(x0)
    acp  = alphas_cumprod[t].view(-1, 1, 1, 1)
    return acp.sqrt() * x0 + (1 - acp).sqrt() * noise


def diffusion_loss(model, x0):
    """
    Sample t ~ Uniform[0, T-1], noise ~ N(0,I),
    corrupt x0 to x_t, predict noise, compute MSE.
    """
    B     = x0.size(0)
    t     = torch.randint(0, T, (B,), device=device)
    noise = torch.randn_like(x0)
    x_t   = q_sample(x0, t, noise)
    t_in  = t.float() / T              # normalise to [0,1]
    pred  = model(x_t, t_in)
    return F.mse_loss(pred, noise)
```

Training loop

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)),   # -> [-1, 1]
])
train_ds     = datasets.MNIST('.', train=True, download=True,
                              transform=transform)
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)

model = SimpleUNet(channels).to(device)
opt   = torch.optim.Adam(model.parameters(), lr=lr)

for epoch in range(epochs):
    total = 0.0
    for x, _ in train_loader:
        x    = x.to(device)
        loss = diffusion_loss(model, x)
        opt.zero_grad(); loss.backward(); opt.step()
        total += loss.item()
    print(f"Epoch {epoch+1}/{epochs}  "
          f"loss={total/len(train_loader):.4f}")
```

Reverse diffusion: sampling

```
@torch.no_grad()
def p_sample_loop(model, shape):
    """DDPM reverse process: x_T -> x_0."""
    x = torch.randn(shape, device=device)   # start from pure noise
    for i in reversed(range(T)):
        t_in    = torch.full((shape[0],), i, device=device).float()/T
        eps_hat = model(x, t_in)            # predict noise

        beta_t = betas[i]
        acp_t  = alphas_cumprod[i]
        coef1  = 1.0 / alphas[i].sqrt()
        coef2  = beta_t / (1.0 - acp_t).sqrt()
        mu     = coef1 * (x - coef2 * eps_hat)

        if i > 0:
            sigma = beta_t.sqrt()
            mu   += sigma * torch.randn_like(x)
        x = mu
    return x

samples = p_sample_loop(model, (16, channels, img_size, img_size))
samples = samples.clamp(-1, 1).cpu()
grid    = torchvision.utils.make_grid(samples, nrow=4, normalize=True)
plt.imshow(grid.permute(1, 2, 0)); plt.axis('off'); plt.show()
```

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

Mathematical parallel: VAE vs. Diffusion

Both optimise a **variational lower bound (ELBO)** on $\log p(\mathrm{x})$.

Property	**VAE**	**DDPM**
Latent dim	$d_h \ll d_x$ (bottleneck)	$d_h = d_x$ (full dim)
# latent levels	1	T (e.g. 1000)
Encoder q	*learned* $q_\phi(\mathrm{h}\|\mathrm{x})$	*fixed* Gaussian
Inference	single forward pass	single forward pass
Decoder p	*learned* $p_\theta(\mathrm{x}\|\mathrm{h})$	*learned* denoiser
Sampling	single decoder call	T denoiser calls
ELBO terms	2 (recon + KL)	T (sum of KL terms)
Training loss	ELBO (two terms)	noise-prediction MSE
Reparameterisation	$\mathrm{h} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$	$\mathrm{x}_t = \sqrt{\bar{\alpha}_t}\mathrm{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$

VAE vs. Diffusion: Pros and Cons

VAE

Pros:

- **Fast** inference and generation (1 pass each)
- **Compact** latent space; useful for downstream tasks (clustering, interpolation)
- **Stable** training with well-understood ELBO
- **Semantic** control via latent arithmetic
- Closed-form KL for Gaussian encoder

Cons:

- **Blurry** samples (Gaussian decoder, ℓ_2 averaging)
- **Posterior collapse** risk

Diffusion (DDPM)

Pros:

- **Sharp, high-quality** samples (state of the art)
- **Stable** training (MSE on noise)
- **No mode collapse** — covers full data distribution
- Theoretically grounded via ELBO and score matching
- Flexible conditioning (text, class, image, audio)

Cons:

- **Slow** sampling ($T = 100$–1000 NFEs)
- **No compact** latent representation
- High memory/compute for large T

When to use which model

Use a VAE when...

- You need a *structured, compressed* latent space for downstream tasks (clustering, regression, anomaly detection, disentanglement).
- Real-time generation or inference is required.
- The data has low intrinsic dimensionality relative to observation space (e.g. molecular graphs, structured scientific data).
- Interpretable uncertainty quantification is important.

Use a Diffusion model when...

- *Sample quality* is the primary goal (images, audio, video).
- Conditioning on complex, high-dimensional signals is needed (text-to-image, super-resolution, inpainting).
- Training stability and diversity are more important than speed.
- You have the compute budget for T-step sampling.

The common ELBO: a unifying view

Both models maximise a lower bound of the same form:

$$\log p(\mathrm{x}) \geq \mathcal{L}(\phi, \boldsymbol{\theta}) = \underbrace{\text{reconstruction likelihood}}_{\mathbb{E}[\log p_{\boldsymbol{\theta}}(\mathrm{x}|z)]} - \underbrace{\text{regularisation}}_{D_{\mathrm{KL}}(q_{\phi} \,\|\, p)}$$

In a VAE, the KL is between encoder and prior over a *single* latent h. In a DDPM, the sum $\sum_{t=1}^{T} \mathcal{L}_{t-1}$ plays the same role: each term is a KL between the forward posterior $q(\mathrm{x}_{t-1}|\mathrm{x}_t, \mathrm{x}_0)$ and the learned reverse $p_{\boldsymbol{\theta}}(\mathrm{x}_{t-1}|\mathrm{x}_t)$, summed over T levels.

Key insight

Diffusion models are VAEs with: (a) T latent levels instead of 1, (b) a fixed encoder instead of a learned one, and (c) full-dimensional latents instead of a bottleneck.

Techniques for faster diffusion sampling

DDPM requires $T \approx 1000$ steps. Several approaches accelerate this:

DDIM (Song et al. 2020)

Replace the stochastic reverse step with a *deterministic* ODE integration. Same noise network, but 50–200× fewer function evaluations (NFEs) for comparable quality.

Latent Diffusion (Rombach et al. 2022)

Run diffusion in a *VAE latent space* instead of pixel space. Combines VAE compression with diffusion quality: lower dimensionality ⇒ fewer NFEs needed. **This is how Stable Diffusion works**.

DPM-Solver (Lu et al. 2022)

Analytical solution of the probability-flow ODE yields high-order methods. As few as 10–20 NFEs for MNIST-quality images.

Consistency Models (Song et al. 2023)

Distill the whole trajectory into a single-step generator; matches diffusion quality with one or two NFEs.

Latent diffusion: VAE meets diffusion

Latent Diffusion Models (LDMs) elegantly combine both worlds:

1. Train a VAE: $\mathrm{x} \to \mathrm{z} \in \mathbb{R}^{d_z}$ with $d_z \ll d_x$.
2. Run DDPM *in the VAE latent space* z.
3. Decode with the VAE decoder: $\hat{\mathrm{z}}_0 \to \hat{\mathrm{x}}$.

Why this works

- The VAE provides a perceptually compressed, semantic representation.
- Diffusion in latent space is much cheaper: $d_z = 4 \times 32 \times 32$ vs. $d_x = 3 \times 256 \times 256$ for Stable Diffusion.
- All conditioning (text, depth, segmentation) is applied in latent space via cross-attention in the U-Net.
- The VAE's KL regularisation ensures the latent space is smooth enough for a diffusion model to learn.

1 Reminder: Variational Autoencoders

2 From Hierarchical VAEs to Diffusion Models

3 Mathematics of Diffusion Models

4 DDPM PyTorch Implementation

5 Comparing VAEs and Diffusion Models

6 Summary and Outlook

Summary: VAEs and Diffusion — key equations

Concept	**VAE**	**DDPM**
Encoder	$q_\phi(\mathrm{h}\|\mathrm{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2 \mathrm{I})$	$q(\mathrm{x}_t\|\mathrm{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathrm{x}_{t-1}, (1-\alpha_t)\mathrm{I})$
Reparam.	$\mathrm{h} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$	$\mathrm{x}_t = \sqrt{\bar{\alpha}_t}\mathrm{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$
Prior	$p(\mathrm{h}) = \mathcal{N}(0, \mathrm{I})$	$p(\mathrm{x}_T) = \mathcal{N}(0, \mathrm{I})$
KL term	$D_{\mathrm{KL}}(q_\phi \\| p)$ (closed form)	$\sum_t D_{\mathrm{KL}}(q(\mathrm{x}_{t-1}\|\mathrm{x}_t, \mathrm{x}_0) \\| p_\theta)$
Loss	$-\mathcal{L} = -\mathbb{E}[\log p] + D_{\mathrm{KL}}$	$\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathrm{x}_t, t)\|^2$
Sampling	$\mathrm{h} \sim q$; one decoder call	$\mathrm{x}_T \sim \mathcal{N}$; T reverse steps

Exercises

Mathematical exercises

1. Verify the closed-form marginal $q(\mathsf{x}_t|\mathsf{x}_0)$ by induction from $q(\mathsf{x}_t|\mathsf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}\mathsf{x}_{t-1}, (1-\alpha_t)\mathsf{I})$.
2. Derive the forward posterior $q(\mathsf{x}_{t-1}|\mathsf{x}_t, \mathsf{x}_0)$ using Bayes' rule and the Gaussian multiplication formula.
3. Show that the denoising-matching ELBO term $\mathcal{L}_{t-1}$ reduces to the noise-prediction MSE when both distributions are Gaussian with equal variances.
4. Compute the closed-form KL divergence for the VAE between $q_\phi(\mathsf{h}|\mathsf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$ and $p(\mathsf{h}) = \mathcal{N}(0, \mathsf{I})$.

Implementation exercises

1. Replace the linear schedule with a cosine schedule and compare sample quality.
2. Implement DDIM: replace the stochastic reverse step with the ODE update and measure speedup vs. quality trade-off.

Plans for next week

Next topics

1. Generative Adversarial Networks (GANs): architecture, training, mode collapse
2. Connecting GANs, VAEs, and Diffusion: a unified perspective on generative models
3. Flow-based models (normalizing flows): exact likelihood, invertible networks
4. Score-based models and stochastic differential equations
5. Discussion of Project 2

Key references for next week

- Goodfellow et al., *Generative Adversarial Nets*, NIPS 2014
- Rezende & Mohamed, *Variational Inference with Normalizing Flows*, ICML 2015
- Song et al., *Score-Based Generative Modeling through SDEs*, ICLR 2021